

MASTER_CONCEPT_V1.0

Master-Konzept: System-Upgrade ABTEILUNG83 (Astro + StudioCMS)

1. Analyse des Status Quo

Die aktuelle Website zeichnet sich durch ein extrem starkes, klares Branding aus:

- **Vibe:** Cyberpunk, Developer-Terminal, "Neonlicht im Regen". Sehr technikaffin, direkt und auf den Punkt.
- **Aktueller Stack:** WordPress kombiniert mit Breakdance/Oxygen und viel Custom CSS. Das ist bereits das obere Ende der Performance-Fahnenstange innerhalb des monolithischen WordPress-Ökosystems.
- **Messaging:** Fokus auf Performance (`status: lighthouse > 90`), sauberen Code, "Zero Bloat" und DSGVO-Konformität.
- **UI/UX:** Horizontale Journeys, klare Typografie, funktionale Elemente. Bestehende UI-Patterns können direkt übernommen werden. Das Design der Pills ist beispielsweise bereits korrekt; es muss lediglich das funktionale Setting im neuen System als Astro-Komponente (`<Pill />`) sauber neu aufgesetzt werden.

2. Der Shift: Von WordPress zu Astro + StudioCMS

Der Wechsel von einem monolithischen Page Builder zu einer nativen Astro-Architektur mit integriertem, selbstgehostetem CMS ist der ultimative Fix für die Positionierung als High-Performance-Agentur.

Die Engine: Astro

- **Zero-JS by Default:** Astro liefert standardmäßig reines HTML aus. Interaktive Elemente (Islands) werden nur geladen, wenn sie zwingend benötigt werden. Das garantiert maximale Lighthouse-Scores out-of-the-box.
- **View Transitions:** Native, nahtlose Seitenübergänge passen ideal zu App-ähnlichen Experiences und dem gewünschten "Neon-Tech"-Look.
- **Komponenten-Architektur:** Volle Kontrolle über das Markup ohne den DOM-Bloat der visuellen Builder.

Das Backend: StudioCMS (Astro-Native)

Die Entscheidung gegen SaaS (Storyblok/Sanity) und für StudioCMS ändert die Spielregeln zugunsten maximaler Datenhoheit und Effizienz:

- **Einheitliche Codebase:** StudioCMS wird als Integration direkt in Astro geladen (`astro add studiocms`). Frontend und Backend leben im selben Repository. Kein Kontextwechsel, keine getrennten Deployments.
- **Selbstgehostete Performance:** Die Datenbank (via Drizzle ORM – SQLite, PostgreSQL oder LibSQL) läuft auf der eigenen Infrastruktur. Keine API-Rate-Limits, keine externen Abhängigkeiten.
- **Blog & Content-Ready:** Nativer Support für strukturierte Inhalte und Markdown/MDX, ideal für den zukünftigen Blog und das Agentur-Portfolio.

3. Strategisches Setup & Execution

Phase 1: Architektur & Datenmodellierung

- **Datenbank-Design:** Übersetzung der bisherigen Custom Post Types (Portfolio, Services) in saubere StudioCMS-Collections.
- **E-Commerce-Erweiterung (Headless):** Produkte und Agentur-Retainer werden im CMS als Content gepflegt. Der Kaufprozess selbst wird ressourcenschonend über einen Headless-Checkout (z.B. Stripe Payment Links oder Lemon Squeezy) abgewickelt. Das hält die Server-Architektur schlank und lagert die komplexe EU-Steuerlogik aus.
- **Styling-Stack:** Astro kombiniert mit Tailwind CSS (oder weiterhin modularem SCSS), um das bestehende Custom CSS in saubere, versionierbare Tokens zu überführen.

Phase 2: Redesign & UI-Evolution

- **Visual Manifest 2.0:** Das Terminal-Thema (`sh start_project.sh`, `commit: design-freeze`) bleibt bestehen, wird aber geschärft. Weg von reinem Text-Terminal hin zu einer hochmodernen Ästhetik.
- **Micro-Interactions:** Nutzung von modernen CSS-Features und performanten Scroll-Animationen, die ohne schweren JS-Overhead (kein jQuery, minimal GSAP) auskommen.
- **Komponenten-Workflow:** Konsequenter Aufbau in wiederverwendbaren Astro-Komponenten (z.B. `<Hero />`, `<TerminalLog />`, `<ProjectCard />`, `<Pill />`).

Phase 3: Entwicklung & Migration

- **Inhaltsmigration:** Export der WP-Datenbankinhalte und Import in die neue StudioCMS-Datenbank.
- **Hybrides Rendering:** Nutzung von Astros SSR (Server-Side Rendering) für dynamische CMS-Routen (Blog, Portfolio) und statischer Generierung für reine Info-Seiten, um die TTFB (Time to First Byte) zu minimieren.
- **Deployment-Pipeline:** Setup einer CI/CD-Pipeline (z.B. über Vercel, Netlify oder einen eigenen VPS via Docker). Jeder Git-Commit triggert einen sauberen Build. Das ist echtes "Deploy ready".

4. Business-Impact des neuen Setups

Mit diesem Stack-Upgrade verändert sich auch das Narrativ der Agentur für zukünftige Kundenprojekte:

1. **Absolute Datenhoheit:** Ein vollständig selbstgehostetes System ohne Vendor-Lock-in durch SaaS-Anbieter. Ein massives Verkaufsargument für datensensible Kunden (DSGVO).
2. **Security by Design:** Keine WordPress-Login-Seite mehr (`/wp-admin`), keine ständigen Plugin-Updates zur Abwehr von Schwachstellen.
3. **Green Web & Performance:** Extrem effiziente Server-Nutzung durch Astros Architektur. Die Seite lädt "schnell wie ein Neuro-Link".

Das Upgrade auf Astro + StudioCMS ist damit die ultimative Einlösung des eigenen Agentur-Codes: "Less Noise. Nice Data. No Bloat."

Tree für Setup:

```
a83-astro/
├── public/                                # Statische Assets (Favicon, Fonts, rohe
SVGs)
│   └── fonts/                            # Lokales Font-Hosting (DSGVO-konform)
├── src/
│   ├── components/                       # UI-Baukasten (Komponenten-Architektur)
│   │   └── ui/                          # "Dumme" Design-Elemente ohne komplexe
Logik
│   │   │   ├── Pill.astro                # Pill-Design (Setting & Props für Astro)
│   │   │   ├── Button.astro
│   │   │   └── GlitchText.astro
│   │   └── blocks/                      # Komplexe Seiten-Sektionen (aus Breakdance
adaptiert)
│   │       ├── HeroTerminal.astro
│   │       ├── ProjectGrid.astro
│   │       └── ServiceCard.astro
│   └── islands/                          # Client-Side JS nur wenn nötig (z.B.
Vue/React/Svelte)
│       └── ContactForm.tsx              # Interaktives Formular mit eigenem State
│   └── layouts/                          # Die Basis-HTML-Gerüste
│       └── BaseLayout.astro             # Das `` und `` Skelett, View
Transitions
│       └── MarkdownLayout.astro         # Spezielles Layout für rechtliche
Seiten/Simpel-Texte
│       └── pages/                       # File-based Routing (Frontend)
│           ├── index.astro              # Homepage
│           └── work/                    # Portfolio
│               ├── index.astro          # Portfolio-Übersicht
│               └── [slug].astro         # Dynamische Route für Einzelprojekte (aus
StudioCMS)
│       └── api/                         # Serverseitige Endpunkte (falls du Webhooks
```

```

brauchst)
|   |─ styles/                # Globale Styles
|   |   └─ global.css         # CSS-Tokens, Custom CSS, Tailwind-Imports
|   └─ utils/                 # Kleine Helfer-Skripte
|       └─ formatDate.ts      # Datum-Formatierung für den Blog
└─ db/                        # Datenbank-Schema (Drizzle ORM für
StudioCMS)
|   └─ schema.ts              # Definition deiner Custom Collections
(Portfolio, Services)
|   └─ seed.ts                # Dummy-Daten für lokale Entwicklung
└─ .env                      # Umgebungsvariablen (DB_URL, Stripe_Keys –
niemals in Git)
└─ astro.config.mjs          # Die Kommandozentrale (Hier wird StudioCMS
eingebunden)
└─ tailwind.config.mjs       # Falls du Tailwind nutzt, sonst deine CSS-
Config
└─ tsconfig.json             # TypeScript-Regeln für sauberen Code
└─ package.json              # Deine Dependencies

```

Entwurf Pill.astro - Single Source of Truth

```

---
// Das funktionale Setting (TypeScript Logic)
interface Props {
  text: string;
  theme?: 'neon' | 'dark' | 'outline';
}

const { text, theme = 'neon' } = Astro.props;

// Tailwind-Klassen basierend auf dem Theme dynamisch zuweisen
const themeClasses = {
  neon: 'bg-green-400 text-black', // (Beispiel für deine Neon-Farbe)
  dark: 'bg-gray-800 text-white',
  outline: 'border border-green-400 text-green-400'
};

---

<span class={`inline-flex items-center px-3 py-1 rounded-full text-sm font-
medium ${themeClasses[theme]}`}>
  {text}
</span>

```

Integrations

1. Native View Transitions (Der App-Vibe)

Astro hat die **View Transitions API** direkt in den Core eingebaut. Das bedeutet, du bekommst nahtlose, SPA-ähnliche (Single Page Application) Seitenübergänge ohne schweres JavaScript-Framework.

- **Wie es funktioniert:** Du importierst `<ViewTransitions />` in dein `BaseLayout.astro` und platzierst es im `<head>`. Astro übernimmt den Rest.
- **Der Agentur-Fit:** Wenn ein User von der Homepage auf ein Portfolio-Projekt klickt, lädt die Seite nicht hart neu. Der Header bleibt stehen, und der Content blendet weich (oder mit einem von dir definierten Terminal-Glitch-Effekt) ein. Das liefert exakt das "Neuro-Link"-Gefühl, das ihr auf der Seite verspricht.

2. Must-Have Integrations für das Agentur-Setup

Um das WordPress-Ökosystem (SEO-Plugins, Caching, Icon-Fonts) vollwertig und performant zu ersetzen, solltest du folgende offizielle oder bewährte Astro-Integrations in dein Setup ziehen:

- **@astrojs/tailwind**
Die offizielle Integration. Sie verknüpft deine `tailwind.config.mjs` direkt mit dem Astro-Build. Klassen werden zur Build-Zeit geparkt und nur das genutzt, was wirklich im Code steht.
- **@astrojs/sitemap**
Ersetzt komplexe SEO-Plugins. Diese Integration generiert bei jedem Build automatisch eine saubere `sitemap.xml` basierend auf deinen Routen (inklusive der dynamischen StudioCMS-Seiten). Ein absolutes Muss für Google.
- **astro-icon**
Das ultimative Tool für SVGs. Anstatt riesige Icon-Bibliotheken wie FontAwesome zu laden (was Performance kostet), erlaubt dir `astro-icon`, SVGs direkt im Code als Komponente aufzurufen (z.B. `<Icon name="mdi:terminal" />`). Es optimiert die SVGs zur Build-Zeit und injiziert nur den reinen SVG-Code ins HTML.
- **@astrojs/partytown**
Eure Geheimwaffe für den `status: lighthouse > 90`. Wenn ihr zukünftig Analytics (Plausible, Google Analytics) oder ein Consent-Management-Tool (Cookie-Banner) ladet, ziehen diese Third-Party-Skripte normalerweise die Performance nach unten. Partytown lagert diese Skripte in einen "Web Worker" aus. Das Haupt-Gewinde (Main Thread) bleibt frei, die Seite lädt instant.
- **astro-seo** (von Jonas Moen)
Ein extrem schlankes Paket, das dir hilft, Meta-Titel, Descriptions, OpenGraph-Images (für Social Media Sharing) und Canonical-URLs sauber in deine Head-Komponente zu injizieren.

Features

Um Premium-Preise abzurufen und richtig Geld zu verdienen, reicht es nicht, "nur" eine schnelle Seite zu haben. Kunden zahlen keine 10.000 Euro für gute Lighthouse-Scores. Sie zahlen für ein digitales Erlebnis, das sich so hochwertig, exklusiv und zukunftssicher anfühlt, dass die Konkurrenz daneben wie ein Relikt aus den 2010er Jahren wirkt.

Um das "Holy Shit"-Aha-Erlebnis zu triggern, das High-Ticket-Sales abschließt, nutzen wir die stärksten Waffen deines neuen Stacks. Hier sind die vier Upgrades für das ultimative Cyberpunk-Terminal-Erlebnis:

1. Die Cmd+K Command Palette (Der pure Hacker-Vibe)

Vergiss klassische Dropdown-Menüs. Wenn jemand deine Seite besucht, sieht er unten rechts einen dezenten Hinweis: `Press Cmd + K to execute`.

Drückt der User die Tastenkombination, öffnet sich ein rasend schnelles, von Tailwind gestyltes Terminal-Overlay (Command Palette).

- **Funktion:** Der User kann tippen: "Preise", "Kontakt", "Dark Mode aktivieren" oder "Portfolio". Die Ergebnisse filtern sich in Echtzeit.
- **Der Wow-Effekt:** Das fühlt sich nicht an wie eine Website, sondern wie eine professionelle IDE (wie VS Code) oder eine High-End-App. Es unterstreicht sofort deine technische Überlegenheit.

2. Der "Speed-to-Cash" ROI-Calculator (Das Astro Island)

Du sagst, deine Seiten sind schneller und bringen mehr Umsatz. Beweise es interaktiv direkt auf der Homepage.

Hier nutzen wir Astros "Islands Architecture". Du baust eine einzige hochinteraktive Komponente (z.B. mit Svelte oder React), während der Rest der Seite statisch bleibt.

- **Funktion:** Ein cleanes, neon-leuchtendes Dashboard mit Slidern. Der Kunde stellt ein: *Aktuelle Besucher* und *Aktuelle Ladezeit*. Der Rechner zeigt in Echtzeit, wie viel Umsatz er durch eine langsame WordPress-Seite monatlich verliert und wie schnell sich dein 5.000€+ Relaunch amortisiert.
- **Der Wow-Effekt:** Du verkaufst kein Design mehr, du verkaufst ein messbares Business-Upgrade. Das rechtfertigt jeden Preis.

3. WebGL Shader / Code-generierte Visuals

Videos und riesige Bilder als Hintergrund sind langsam und langweilig. Für das echte "Neonlicht im Regen"-Gefühl nutzt du WebGL (z.B. mit Three.js).

- **Funktion:** Ein dunkler Canvas-Hintergrund, auf dem ein dezenter, interaktiver Matrix-Regen, ein fließendes Neon-Grid oder ein Partikel-System läuft, das auf Mausbewegungen reagiert.
- **Der Wow-Effekt:** Da es direkt im Browser durch Code berechnet wird, ist es gestochen scharf, extrem flüssig (60fps) und lädt in Millisekunden. Es zeigt, dass du "Deep Tech" beherrscht und nicht nur Templates zusammenklickst.

4. Dezent UI Sound Design (Das Apple-Prinzip)

Das menschliche Gehirn reagiert extrem stark auf multisensorische Reize.

- **Funktion:** Ein winziges, kaum hörbares mechanisches "Klick"-Geräusch, wenn man über deine Pills oder Buttons hovers. Ein tiefer, satter "Swoosh", wenn eine Seite über die View Transitions wechselt.
- **Der Wow-Effekt:** Es macht die digitale Interaktion physisch spürbar. Web-Apps, die Sound Design richtig einsetzen, wirken psychologisch sofort doppelt so teuer.

Skizze Cmd+K Command Palette

Um dem "Zero Bloat"-Mantra deiner Agentur absolut treu zu bleiben, verzichten wir hier auf schwere JavaScript-Bibliotheken (wie React oder Vue). Wir nutzen stattdessen das native HTML5 `<dialog>`-Element kombiniert mit Vanilla JavaScript direkt in Astro. Das ist rasend schnell, out-of-the-box barrierefrei und erzeugt null Overhead.

Der Code: `src/components/islands/CommandPalette.astro`

```
----
// Die Links/Befehle, die der User durchsuchen kann.
// Später kannst du diese Liste dynamisch aus dem StudioCMS füttern!
const commands = [
  { title: "Startseite", url: "/", icon: "home", type: "Page" },
  { title: "Portfolio ansehen", url: "/work", icon: "folder", type: "Work"
},
  { title: "Projekt anfragen", url: "/contact", icon: "terminal", type:
"Action" },
  { title: "Preise & Module", url: "/services", icon: "credit-card", type:
"Service" },
  { title: "Astro vs WordPress", url: "/blog/astro-wp", icon: "file-text",
type: "Blog" }
];
----

<dialog
  id="cmd-palette"
  class="backdrop:bg-black/80 bg-gray-950 border border-green-500/30
rounded-xl text-gray-300 w-full max-w-2xl shadow-
[0_0_40px_rgba(34,197,94,0.15)] p-0 overflow-hidden font-mono"
>
  <div class="flex flex-col">
    <div class="flex items-center px-4 border-b border-gray-800/50 bg-gray-
900/50">
      <span class="text-green-500 mr-3 animate-pulse">_</span>
      <input
```

```

        type="text"
        id="cmd-input"
        placeholder="System durchsuchen... (z.B. 'Portfolio')"
        class="w-full bg-transparent py-5 outline-none text-green-400
placeholder-gray-700 text-lg"
        autocomplete="off"
    />
    <kbd class="hidden sm:inline-block text-xs text-gray-500 border
border-gray-800 rounded px-2 py-1 bg-gray-900">ESC</kbd>
</div>

<ul id="cmd-results" class="max-h-[60vh] overflow-y-auto p-2">
    {commands.map((cmd) => (
        <li>
            <a
                href={cmd.url}
                class="cmd-item flex items-center justify-between p-3 rounded-lg
hover:bg-green-500/10 hover:text-green-400 transition-colors group cursor-
pointer"
                data-title={cmd.title.toLowerCase()}
            >
                <div class="flex items-center gap-3">
                    <span class="text-gray-500 group-hover:text-green-500">
[{{cmd.type}}]</span>
                    <span>{{cmd.title}}</span>
                </div>
                <span class="text-gray-600 text-sm group-hover:text-green-500
transition-colors opacity-0 group-hover:opacity-100">
EXECUTE ↵
                </span>
            </a>
        </li>
    )})
</ul>

<div class="px-4 py-3 border-t border-gray-800/50 text-xs text-gray-600
flex justify-between bg-black/20">
    <span>ABTEILUNG83 // KERNEL V2.0</span>
    <span>Navigation aktiv</span>
</div>
</div>
</dialog>

```

```

<script>
  // Vanilla JS – wird von Astro nur einmal geladen und gebündelt
  const dialog = document.getElementById('cmd-palette') as
HTMLDialogElement;
  const input = document.getElementById('cmd-input') as HTMLInputElement;
  const items = document.querySelectorAll('.cmd-item');

  // Tastenkombination abfangen (Cmd+K auf Mac, Ctrl+K auf Windows)
  document.addEventListener('keydown', (e) => {
    if ((e.metaKey || e.ctrlKey) && e.key === 'k') {
      e.preventDefault();
      if (dialog.open) {
        dialog.close();
      } else {
        dialog.showModal();
        input.value = ''; // Reset input
        filterResults(''); // Reset filter
        input.focus();
      }
    }
  });

  // Klick auf Backdrop schließt das Modal
  dialog.addEventListener('click', (e) => {
    const dialogDimensions = dialog.getBoundingClientRect();
    if (
      e.clientX < dialogDimensions.left || e.clientX >
dialogDimensions.right ||
      e.clientY < dialogDimensions.top || e.clientY >
dialogDimensions.bottom
    ) {
      dialog.close();
    }
  });

  // Filter-Logik
  const filterResults = (searchTerm: string) => {
    items.forEach((item) => {
      const el = item as HTMLElement;
      const title = el.getAttribute('data-title') || '';
      if (title.includes(searchTerm.toLowerCase())) {
        el.parentElement!.style.display = 'block';
      } else {

```

```

        el.parentElement!.style.display = 'none';
    }
});
};

input.addEventListener('input', (e) => {
    filterResults((e.target as HTMLInputElement).value);
});
</script>

```

Warum das für 10k+ Budgets sorgt:

1. **Die Experience:** Der User drückt `Cmd+K`, der Hintergrund verdunkelt sich mit einem satten Blur (`backdrop:bg-black/80`), und ein leuchtendes Terminal-Eingabefeld erscheint sofort (ohne Ladezeit).
2. **Die Geschwindigkeit:** Da die Links bereits im DOM liegen, filtert das JavaScript die Liste in absoluter Echtzeit – beim Tippen jedes einzelnen Buchstabens.
3. **Die Integration:** Diese Komponente bindest du einfach einmalig in dein `BaseLayout.astro` ein, und sie ist sofort auf jeder Unterseite verfügbar.

2. Der "Speed-to-Cash" ROI-Calculator (Das Astro Island)

Hier ist die Skizze für den **Speed-to-Cash ROI-Calculator**.

Anstatt deinen Kunden in stundenlangen Meetings zu erklären, warum Ladezeit wichtig ist, lässt du sie den Schmerz (verlorener Umsatz) und die Lösung (dein Astro-Upgrade) selbst berechnen.

Auch hier bleiben wir dem "Zero Bloat"-Ansatz treu: Wir brauchen kein schweres React. Ein nativer HTML-Slider und ein paar Zeilen Vanilla JavaScript in einer Astro-Komponente reichen völlig aus, um ein flüssiges, reaktives Dashboard zu bauen.

Der Code: `src/components/islands/RoiCalculator.astro`

```

---
// Zero Bloat: Keine externen Libraries, reines HTML/Tailwind
---

<div class="bg-gray-950 border border-gray-800 rounded-xl p-8 font-mono
shadow-[0_0_30px_rgba(0,0,0,0.5)] max-w-3xl mx-auto relative overflow-
hidden">

    <div class="flex items-center justify-between mb-8 border-b border-gray-
800 pb-4">
        <div class="flex gap-2">
            <div class="w-3 h-3 rounded-full bg-red-500/50"></div>

```

```

    <div class="w-3 h-3 rounded-full bg-yellow-500/50"></div>
    <div class="w-3 h-3 rounded-full bg-green-500"></div>
  </div>
  <span class="text-gray-500 text-sm">/sys/calc/roi_simulator.sh</span>
</div>

<div class="grid grid-cols-1 md:grid-cols-2 gap-12">

  <div class="space-y-8">
    <div>
      <label class="flex justify-between text-gray-400 text-sm mb-2">
        <span>Besucher pro Monat</span>
        <span id="val-visitors" class="text-green-400 font-
bold">10.000</span>
      </label>
      <input type="range" id="slide-visitors" min="1000" max="100000"
step="1000" value="10000"
        class="w-full accent-green-500 cursor-pointer bg-gray-800 h-1
rounded-full appearance-none outline-none" />
    </div>

    <div>
      <label class="flex justify-between text-gray-400 text-sm mb-2">
        <span>Wert pro Lead/Kunde (€)</span>
        <span id="val-value" class="text-green-400 font-bold">500 €</span>
      </label>
      <input type="range" id="slide-value" min="50" max="5000" step="50"
value="500"
        class="w-full accent-green-500 cursor-pointer bg-gray-800 h-1
rounded-full appearance-none outline-none" />
    </div>

    <div>
      <label class="flex justify-between text-gray-400 text-sm mb-2">
        <span>Aktuelle Ladezeit (Sekunden)</span>
        <span id="val-speed" class="text-red-400 font-bold">4.5 s</span>
      </label>
      <input type="range" id="slide-speed" min="1" max="10" step="0.5"
value="4.5"
        class="w-full accent-red-500 cursor-pointer bg-gray-800 h-1
rounded-full appearance-none outline-none" />
    </div>
  </div>
</div>

```

```

    <div class="bg-gray-900 border border-gray-800 rounded-lg p-6 flex flex-
col justify-center relative">
    <div class="absolute -inset-1 bg-green-500/20 blur-xl rounded-lg z-0
opacity-0 transition-opacity duration-500" id="success-glow"></div>

    <div class="relative z-10">
        <p class="text-gray-500 text-xs mb-1 uppercase tracking-
wider">Verlorener Umsatz (p.a.)</p>
        <div class="text-red-400 text-3xl font-bold mb-6 line-through
decoration-red-900/50" id="out-loss">
            - 35.000 €
        </div>

        <p class="text-gray-500 text-xs mb-1 uppercase tracking-
wider">Potenzial mit Astro-Upgrade (p.a.)</p>
        <div class="text-green-400 text-5xl font-bold mb-2 tracking-tighter"
id="out-gain">
            + 24.500 €
        </div>
        <p class="text-gray-400 text-xs mt-4 border-t border-gray-800 pt-4">
            *Basierend auf Google-Daten: 0.1s schnellere Ladezeit = ~1% mehr
Conversion. Ziel-Ladezeit: < 0.8s.
        </p>
    </div>
</div>
</div>
</div>
</div>

<script>
    // Die Business-Logik (Vanilla JS)
    const visitorsSlide = document.getElementById('slide-visitors') as
HTMLInputElement;
    const valueSlide = document.getElementById('slide-value') as
HTMLInputElement;
    const speedSlide = document.getElementById('slide-speed') as
HTMLInputElement;

    const valVisitors = document.getElementById('val-visitors');
    const valValue = document.getElementById('val-value');
    const valSpeed = document.getElementById('val-speed');

    const outLoss = document.getElementById('out-loss');

```

```

const outGain = document.getElementById('out-gain');
const glow = document.getElementById('success-glow');

// Formatierung für Euro
const formatEUR = (num: number) => new Intl.NumberFormat('de-AT', { style:
'currency', currency: 'EUR', maximumFractionDigits: 0 }).format(num);

const calculateROI = () => {
  const visitors = parseInt(visitorsSlide.value);
  const leadValue = parseInt(valueSlide.value);
  const currentSpeed = parseFloat(speedSlide.value);

  // Astro Ziel-Ladezeit ist konservativ 0.8 Sekunden
  const targetSpeed = 0.8;
  const speedDiff = currentSpeed - targetSpeed;

  // UI Updates für die Slider-Werte
  valVisitors!.innerText = visitors.toLocaleString('de-AT');
  valValue!.innerText = formatEUR(leadValue);
  valSpeed!.innerText = currentSpeed.toFixed(1) + ' s';

  if (currentSpeed > 1.5) {
    valSpeed!.className = 'text-red-400 font-bold';
  } else {
    valSpeed!.className = 'text-green-400 font-bold';
  }

  // Die Formel: Jede gesparte Sekunde bringt ca. 10% mehr Conversion
  (Google Benchmark)
  // Angenommene Basis-Conversion-Rate: 2%
  const baseConversion = 0.02;
  const currentConversions = visitors * baseConversion;

  // Verlustberechnung
  const conversionPenalty = (currentSpeed - 1) * 0.10; // 10% penalty per
second over 1s
  const lostConversions = currentConversions * conversionPenalty;
  const yearlyLoss = Math.max(0, lostConversions * leadValue * 12);

  // Astro-Gewinn
  const gainedConversions = lostConversions * 0.9; // 90% des Verlustes
durch Astro zurückholen
  const yearlyGain = Math.max(0, gainedConversions * leadValue * 12);

```

```

// Ausgabe
outLoss!.innerText = '- ' + formatEUR(yearlyLoss);
outGain!.innerText = '+ ' + formatEUR(yearlyGain);

// Visuelles Feedback
if (yearlyGain > 10000) {
  glow!.classList.remove('opacity-0');
  glow!.classList.add('opacity-100');
} else {
  glow!.classList.add('opacity-0');
  glow!.classList.remove('opacity-100');
}
};

// Event Listener an alle Slider hängen
[visitorsSlide, valueSlide, speedSlide].forEach(input => {
  input.addEventListener('input', calculateROI);
});

// Initiale Berechnung beim Laden
calculateROI();
</script>

```

Warum das für dich als Agentur alles verändert:

1. **Der Frame-Shift:** Wenn ein Kunde sieht, dass ihn seine langsame Seite *jedes Jahr 24.000 € kostet*, ist dein Angebot von 8.000 € für den Astro-Relaunch plötzlich kein "Kostenpunkt" mehr, sondern ein "No-Brainer" mit sofortigem Return on Investment.
2. **Die Gamification:** Menschen lieben es, an Slidern zu spielen. Während sie ihre eigenen Zahlen eingeben, visualisieren sie bereits den Erfolg. Das ist das stärkste Verkaufspsychologie-Tool, das du auf einer Landingpage haben kannst.
3. **Tech-Proof:** Du erklärst nicht nur, dass Astro dynamisch und schnell ist – du demonstrierst es. Die Zahlen updaten sich beim Schieben in Echtzeit, komplett ohne Ruckeln, im echten Terminal-Look.

3. WebGL Shader / Code-generierte Visuals

Das ist der entscheidende Punkt. Hier trennt sich die Spreu vom Weizen.

Jeder kann ein Video von pexels.com in den Hintergrund legen. Aber ein **WebGL Shader** ist etwas völlig anderes. Es ist kein Video. Es ist lebendiger Mathematik-Code, der 60 Mal pro Sekunde auf der Grafikkarte (GPU) des Besuchers ausgeführt wird.

Das Konzept: "Digital Rain / Neon Refraction"

Wir nehmen deinen Claim "Wie Neonlicht im Regen" wörtlich und übersetzen ihn in Code.

Stell dir vor, du schaust nachts durch eine regennasse Fensterscheibe auf eine Cyberpunk-Stadt.

- **Der Vibe:** Dunkel, atmosphärisch, technisch.
- **Der visuelle Effekt:** Digitale "Regentropfen" (eher wie Datenströme oder Glitches) laufen den Bildschirm hinunter. Dahinterliegende neon-grüne Lichtquellen werden durch diese Tropfen gebrochen und verzerrt.
- **Die Interaktion:** Wenn der User die Maus bewegt, beeinflusst er diesen Fluss. Der Mauszeiger wird zu einer Lichtquelle, die die digitalen Regentropfen um sich herum aufleuchten lässt und bricht. Es fühlt sich an, als würde man mit dem Finger über eine beschlagene Scheibe fahren, hinter der ein Serverrack leuchtet.

Warum das den "Holy Shit"-Effekt auslöst:

1. **Es ist ultrascharf:** Egal ob auf einem alten Laptop oder einem 5K Retina Display – der Shader wird immer in der nativen Auflösung des Geräts berechnet. Es gibt keine Pixelartefakte wie bei Videos.
2. **Es ist performant (Zero Bloat):** Wir nutzen dafür **kein Three.js** (das wären ~600kb extra JavaScript). Wir schreiben einen "rohen" Fragment Shader in GLSL (OpenGL Shading Language). Das sind vielleicht 4kb Code, der direkt auf der GPU läuft. Das ist die absolute Königsklasse der Web-Performance.
3. **Es zeigt technische Dominanz:** Wer so etwas "from scratch" baut, zeigt dem Kunden unterbewusst: "Wir klicken keine Templates zusammen. Wir beherrschen die Matrix."

Die technische Umsetzung in Astro

Wir kapseln diese komplexe Technik in einer einzigen Astro-Insel. Diese Komponente wird im Hero-Bereich hinter den Text gelegt.

Die Architektur der Komponente (`src/components/islands/NeonRainBackground.astro`):

```
<canvas id="gl-canvas" class="fixed inset-0 w-full h-full -z-10 pointer-events-none"></canvas>

<script>
  // Das hier ist der "Boilerplate"-Code, um WebGL zu starten.
  // Er ist etwas technisch, aber notwendig, um ohne Libraries zu arbeiten.

  const canvas = document.getElementById('gl-canvas') as HTMLCanvasElement;
  const gl = canvas.getContext('webgl');

  if (!gl) {
    console.error("WebGL not supported, falling back to static image.");
```

```

// Hier könnte man ein statisches Fallback-Bild setzen
return;
}

// --- DER KERN: Der Fragment Shader (GLSL Code) ---
// Das ist die Magie. Dieser String wird auf die Grafikkarte geschickt.
const fragmentShaderSource = `
    precision mediump float;

    uniform float u_time;          // Die Zeit (für Animation)
    uniform vec2 u_resolution;     // Bildschirmgröße
    uniform vec2 u_mouse;          // Mausposition

    // Eine Funktion, um "Rauschen" (Noise) zu erzeugen – für den Regen-Look
    float random (in vec2 _st) {
        return fract(sin(dot(_st.xy, vec2(12.9898,78.233))) *
43758.5453123);
    }

    void main() {
        // Normalisierte Pixel-Koordinaten (von 0.0 bis 1.0)
        vec2 uv = gl_FragCoord.xy / u_resolution.xy;

        // --- DER EFFEKT (stark vereinfachtes Konzept) ---

        // 1. Regen-Simulation
        // Wir verzerren die UV-Koordinaten basierend auf Zeit und Noise,
        // um den Effekt von herablaufendem Wasser zu erzeugen.
        vec2 distortedUV = uv;
        distortedUV.y += u_time * 0.1; // Bewegung nach unten
        float rainNoise = random(floor(distortedUV * vec2(20.0, 1.0))); //
Vertikale Streifen

        // 2. Neon-Lichtquelle (Maus)
        // Berechne Abstand des aktuellen Pixels zur Maus
        float mouseDist = distance(uv, u_mouse);
        // Erzeuge einen leuchtenden Kreis um die Maus
        float mouseGlow = 1.0 - smoothstep(0.0, 0.3, mouseDist);

        // 3. Farben mischen
        vec3 bgColor = vec3(0.05, 0.05, 0.08); // Fast Schwarz
        vec3 neonColor = vec3(0.0, 1.0, 0.4); // Das ABTEILUNG83 Grün

```

```

        // Das Licht wird durch den "Regen" gebrochen
        vec3 finalColor = bgColor + (neonColor * mouseGlow * rainNoise *
0.5);

        gl_FragColor = vec4(finalColor, 1.0);
    }
};

// --- WebGL Setup (gekürzt für die Übersicht) ---
// Hier würde der Code stehen, der den Shader kompiliert,
// die Daten (Zeit, Mausposition) an die GPU sendet und
// den Loop (requestAnimationFrame) startet.
// ...

// Resize Handler damit es immer scharf bleibt
function resize() {
    canvas.width = window.innerWidth;
    canvas.height = window.innerHeight;
    // Update u_resolution uniform...
}
window.addEventListener('resize', resize);
resize();
</script>

```

Das Ergebnis: Ein atemberaubender, interaktiver Hintergrund, der weniger als 10KB wiegt und auf jeder Hardware flüssig läuft. Das ist die ultimative Visitenkarte für eine High-Performance-Agentur.

4. Dezentess UI Sound Design (Das Apple-Prinzip)

Der vierte und letzte Baustein für das "Holy Shit"-Erlebnis. Sound im Webdesign ist ein schmaler Grat: Wenn man es falsch macht, ist es unfassbar nervig (denk an die Flash-Websites der frühen 2000er). Wenn man es aber richtig macht, transformiert es eine Website in eine **Software-Experience**.

Apple, Tesla, Teenage Engineering – alle High-End-Tech-Brands nutzen mikroskopisch kurzes Sound-Feedback, um Wertigkeit zu vermitteln. Für den Cyberpunk/Terminal-Vibe von Abteilung83 ist das ein absoluter Gamechanger.

Das Konzept: Taktils Audio-Feedback

Wenn ein User mit deiner Seite interagiert, muss es sich anfühlen, als würde er echte, physische Schalter an einem teuren Synthesizer oder einer mechanischen Tastatur bedienen.

- **Der Hover-Sound (Pills & Buttons):** Ein extrem kurzes, tiefes "Tick" (unter 50 Millisekunden). Kein lauter Ton, eher ein Klicken, das man fast mehr spürt als hört.

- **Der Action-Sound (Cmd+K oder Formular absenden):** Ein satterer, mechanischer "Clack"-Sound, ähnlich dem Drücken der Enter-Taste auf einem Cherry-MX-Keyboard, gefolgt von einem leisen, digitalen Bestätigungs-Piep.
- **Der Transition-Sound:** Wenn Astro die View Transitions feuert (der nahtlose Seitenwechsel), ein sehr tiefer, unterschwelliger "Swoosh" oder ein leises elektrisches Summen.

Die technische Umsetzung (Zero Bloat)

Auch hier gilt: Keine schweren Libraries wie Howler.js. Wir nutzen reines Vanilla JavaScript und winzige, hochkomprimierte `.webm` oder `.mp3` Dateien (je nur 1-2 KB groß), die wir asynchron vorladen, damit es beim Klicken keine Ladeverzögerung gibt.

Die Architektur (`src/utils/soundDesign.ts`):

Wir lagern die Logik in ein kleines Utility-Script aus. Das hält deine Astro-Komponenten sauber.

```
// Ein simples, performantes Audio-System ohne externe Libraries

// Audio-Dateien vorladen (liegen im /public Ordner)
const sounds = {
  hover: new Audio('/sounds/tick.webm'),
  click: new Audio('/sounds/clack.webm'),
  success: new Audio('/sounds/terminal-beep.webm')
};

// Lautstärke extrem leise einstellen (es soll subliminal sein)
Object.values(sounds).forEach(audio => {
  audio.volume = 0.15; // 15% Lautstärke
});

let isMuted = false; // Optional: User kann Sound muten

export const playSound = (type: keyof typeof sounds) => {
  if (isMuted) return;

  const sound = sounds[type];
  // Audio zurücksetzen, falls es noch spielt (für schnelles Hovern)
  sound.currentTime = 0;

  // Try-Catch fängt Browser-Einschränkungen ab
  // (Browser blockieren manchmal Audio ohne vorherige User-Interaktion)
  sound.play().catch(() => {
    // Silent fail – wenn der Browser es blockiert, passiert einfach nichts.
    // Kein Crash.
  });
};
```

```
};

export const toggleMute = () => {
  isMuted = !isMuted;
};
```

Die Integration in deine Komponenten

Erinnerst du dich an deine `Pill.astro` oder deine Buttons? Du importierst das Script einfach dort, wo du es brauchst:

```
---
// In deiner Button.astro oder Projektkarte
---
<a
  href="/work"
  class="interactive-element group inline-flex ..."
  data-sound="hover"
>
  Projekt ansehen
</a>

<script>
  import { playSound } from '../utils/soundDesign.ts';

  // Wir suchen alle Elemente mit einem Sound-Tag und hängen Listener an
  const elements = document.querySelectorAll('.interactive-element');

  elements.forEach(el => {
    el.addEventListener('mouseenter', () => playSound('hover'));
    el.addEventListener('click', () => playSound('click'));
  });
</script>
```

Der "Hacker"-Bonus

Damit das Ganze professionell bleibt, baust du unten im Footer (oder in deiner Cmd+K Palette) einen kleinen Schalter ein: `[ ] Audio Feedback aktivieren`.

Wenn der User die Seite betritt, ist der Sound aus rechtlichen/UX-Gründen oft vom Browser blockiert. Sobald er das erste Mal klickt (z.B. in der Command Palette), entsperrt der Browser das Audio-System und die Magie beginnt.
